

AI-Native Workflow

Hierarchical Intent, Evidence-Gated Verification, and Dependency-Local Recovery for
Long-Horizon Language-Agent Execution

Yuhe Sui

Version 5.6.35 — 30 August 2026

Abstract

Long-horizon language-agent work fails less often because a model cannot generate a plausible next step than because intent, authority, evidence, and dependencies drift across many steps and contexts. This report specifies a package-resident control layer whose canonical hierarchy is *Plan* $\rightarrow$ *Phase* $\rightarrow$ *durable intent contract (DIC)* $\rightarrow$ *disposable execution prompt stack (EPS)* $\rightarrow$ *Prompt/Run* $\rightarrow$ *Action*. A human approves a multi-phase Plan and reviews periodic or material checkpoints; Main, Theory, and Experiment workstreams autonomously manage ordinary phases. We formalize semantic refinement, contract composition, verification outcomes, dependency-local invalidation, imperfect verifiers, context projection, and human-checkpoint complexity. Under complete read/write declarations and versioned evidence, a repair need invalidate only its dynamic dependency cone, and that cone is worst-case tight. Under a conditional verifier miss bound $\alpha < 1$, an undetected fault survives k gates with probability at most α^k ; a stationary cost model yields an explicit checkpoint-interval trade-off. Conversely, if observed evidence aliases goal-satisfying and goal-violating states, no evidence-only verifier can avoid both false acceptance and false rejection. These results clarify the mechanism and its limits; they do not establish that hierarchy alone is novel or empirically superior. We therefore freeze a resource-constrained five-task, fourteen-run workshop proof of concept comparing a competent flat ChatGPT–Claude Code workflow with the full package-resident protocol, while preserving the broader matched-budget horizon/fault programme as future work.

Contents

1	Introduction	6
2	The Long-Horizon Agent Problem	6
2.1	Execution traces and drift	6
2.2	Characteristic failures	7
3	Hierarchical Execution Model	7
3.1	Levels and roles	7
3.2	Plans and phases	8
3.3	Durable intent contracts	8
3.4	EPS, prompt/runs, and actions	8
4	Intent and Contract Refinement	8
4.1	Trace semantics	8
4.2	Acceptance predicates and acceptable-state sets	9
4.3	Compositional satisfaction	9

5	Verification and Replanning	9
5.1	Closed-loop operator	9
5.2	Imperfect verification	10
5.3	Repair versus replan	10
6	Dependency Locality and Failure Propagation	10
6.1	Declared footprints	10
6.2	Propagation bounds	11
7	Context and Persistent State	11
7.1	Package as authority	11
7.2	Context projection	11
8	Human Oversight Across Timescales	12
8.1	Governance complexity	12
8.2	Checkpoint interval under imperfect verification	12
9	Main, Theory, and Experiment Specialization	13
10	Reference Implementation and Repository Architecture	13
10.1	Authority hierarchy and source-of-truth policy	13
10.2	Canonical package layout	13
10.3	Plan records	14
10.4	Phase records and lifecycle fields	14
11	The Multi-View Durable Intent Contract	15
11.1	DIC manifest and package semantics	15
11.2	README.md: orientation and recovery view	15
11.3	REQUESTS.md: specific goals and acceptance inventory	15
11.4	PLAN.md: durable execution graph	16
11.5	Parallelism and subagent declarations	16
11.6	ARCHITECTURE.md: target repository/system architecture	17
11.7	TEST_MATRIX.md: concrete test or experiment matrix	17
11.8	Custom DIC views	17
12	Execution Prompt Stack and Prompt/Run Lifecycle	18
12.1	DIC Plan versus EPS	18
12.2	EPS representation	18
12.3	Prompt/run materialization	18
12.4	Bounded subagent contract	18
12.5	Parallel execution and integration	19
12.6	Retries, repair nodes, and supersession	19
13	Phase Execution Lanes, Evidence, and Reports	19
13.1	Core execution surfaces	19
13.2	Optional lanes and <code>off auto on</code>	19
13.3	Minor lane	20
13.4	Research lane without a persistent Researcher role	20
13.5	Verification lane	20
13.6	Checkpoint lane	20
13.7	Mandatory compact <code>reports/</code> surface	20
13.8	Merged-raw semantics	21

14 Operational Execution Protocol	21
14.1 Step 1: initialize and recover package state	21
14.2 Step 2: create the Plan and Phase graph	21
14.3 Step 3: approve the Plan	22
14.4 Step 4: construct the Phase DIC package	22
14.5 Step 5: configure lanes and report policy	22
14.6 Step 6: start the Phase and recover its read set	22
14.7 Step 7: instantiate the next EPS batch	23
14.8 Step 8: execute prompts and preserve raw evidence	23
14.9 Step 9: local verification throughout execution	23
14.10 Step 10: milestone review after meaningful clusters	23
14.11 Step 11: interpret verification outcomes	23
14.12 Step 12: handle faults by dependency-local repair	24
14.13 Step 13: maintain Requests and reports during the Phase	24
14.14 Step 14: independent review at material boundaries	24
14.15 Step 15: Phase closure	24
14.16 Step 16: periodic human checkpoint	24
15 Role Specialization, Routing, and Execution Intensity	25
15.1 Main	25
15.2 Theory	25
15.3 Experiment	25
15.4 Model routing	25
15.5 Execution intensity and proportionality	25
16 Verification, Replanning, and Human Governance in Operation	26
16.1 Immediate material escalation categories	26
16.2 What does not normally require escalation	26
16.3 Repair versus replan	26
16.4 Checkpoint semantics	26
17 State Recovery, Context Construction, and Handoffs	26
17.1 Fresh-session recovery order	26
17.2 Context projection policy	27
17.3 Handoff quality	27
18 CLI and Configuration Reference	27
18.1 Hierarchical runtime commands	27
18.2 Core configuration	27
18.3 Compatibility CLI	28
19 Migration and Backward Compatibility	28
20 Conformance, Validation, and Failure Policy	28
20.1 Engineering conformance coverage	28
20.2 Report validation	29
20.3 Failure retention	29
20.4 Resource and external-action policy	29

21 Worked Phase Profiles	29
21.1 Theory Phase profile	29
21.2 Experiment Phase profile	29
21.3 Runtime reconstruction Phase profile	30
21.4 Publication Phase profile	30
22 Detailed End-to-End Example	30
23 Formal Properties and Limits	31
23.1 What is guaranteed by construction	31
23.2 Observability limit	31
23.3 Open-world and adversarial limits	31
23.4 Novelty limit	31
24 Empirical Hypotheses and Evaluation Design	31
24.1 Workshop-stage objective and evidence standard	31
24.2 Model/account architecture and contamination boundary	32
24.3 Primary comparison: Flat versus Workflow	32
24.4 Five-task frozen workshop set	33
24.5 Fourteen-run schedule	33
24.6 Outcomes	33
24.7 Mechanism-oriented expectations	34
24.8 Execution phases and evidence freeze	34
24.9 Current evidence boundary	34
25 Related Work	34
26 Limitations	35
27 Conclusion	35
A Claim classification summary	36
B Required experiment record	36
C Ordinary execution brief	36
D Operational Checklists	36
D.1 Before starting a Phase	37
D.2 Before delegating a subagent prompt	37
D.3 Before accepting a theory claim	37
D.4 Before freezing an experiment	38
D.5 Before a destructive or external action	38
D.6 Before closing a Phase	38
E Prompt-Graph Conventions	38
E.1 Stable node IDs	38
E.2 Recommended node fields	38
E.3 Parallel batch example	39
E.4 Implementation batch example	39

F	Report-Surface Reference Profiles	39
F.1	Theory	40
F.2	Experiment	40
F.3	Software/runtime	40
F.4	Publication/release	40
G	State and Lineage Field Guide	40
G.1	Plan-level state	40
G.2	Phase-level state	41
G.3	Lineage interpretation	41
H	Failure-Response Matrix	41
I	Compatibility Map for v5.6.31 Concepts	42
J	Example Bounded Prompts	42
J.1	Theory derivation node	42
J.2	Experiment run node	43
J.3	Independent release audit node	43

1 Introduction

Language agents increasingly edit repositories, run tools, synthesize evidence, and coordinate other agents. The difficult regime is not a single tool call but a programme extending across many dependent decisions, artifacts, model contexts, and human review cycles. In that regime, a locally fluent response can coexist with global failure: the original objective is weakened, an assumption silently changes, an intermediate artifact becomes stale, a verifier checks the wrong object, or a late repair forces unnecessary re-execution.

Existing agents already plan, reflect, search, use memory, and communicate in manager–worker arrangements (Yao et al., 2023b,a; Besta et al., 2024; Shinn et al., 2023; Wu et al., 2023; Hong et al., 2024). The claim of this report is therefore deliberately narrower. Long-horizon execution benefits from an explicit *authority and refinement protocol*: persistent intent is approved at a human timescale, refined through bounded machine contracts, linked to evidence, and repaired at the smallest dependency-valid scope. The protocol must survive a fresh session with no access to the original conversation.

AI-Native Workflow 5.6.35 implements that protocol inside the working package. Its small authority index is `.ai-workflow/STATE.json`; full contracts, operational refinements, action records, and evidence pointers are package-resident. The default persistent workstreams are Main, Theory, and Experiment. Orchestration is a capability of the active workstream rather than a fourth chat. Human governance is separated from machine execution: a Plan is approved once, ordinary phases proceed autonomously, periodic checkpoints aggregate several phases, and material changes escalate immediately.

The report makes four contributions. First, it gives an executable specification of the hierarchy and state machine. Second, it formalizes what intent preservation and compositional satisfaction would require rather than treating hierarchy as self-justifying. Third, it proves a dependency-local recovery theorem, an imperfect-verification bound, a checkpoint-cost proposition, and an observability impossibility. Fourth, it freezes a targeted workshop evaluation that compares a competent flat ChatGPT–Claude Code workflow against the full package-resident protocol on five evolving/stateful/scientific task families, while retaining the larger matched-budget factorial study for future work. The formal results are adaptations of established contract, incremental-computation, reliability, and partial-observability ideas; the intended scientific contribution is their integrated operationalization and empirical test in language-agent harnesses.

Claim discipline. Every substantial statement is assigned one of: definition; invariant by construction; theorem or proposition under explicit assumptions; empirical hypothesis; conjecture; or unsupported and removed. The machine-readable ledger in `research/CLAIM_LEDGER.json` is authoritative for manuscript integration.

2 The Long-Horizon Agent Problem

2.1 Execution traces and drift

Let $\mathcal{S}$ be the joint repository–environment state, including files, tool state, external observations that have been durably recorded, and governance metadata. A primitive action a induces a possibly stochastic transition kernel

$$K_a : \mathcal{S} \rightsquigarrow \mathcal{S} \times \mathcal{E},$$

where $\mathcal{E}$ contains logs, test outputs, citations, measurements, or human decisions. A run is a trace $\tau = (s_0, a_1, e_1, s_1, \dots, a_T, e_T, s_T) \in \mathcal{T}$. A global intent is not merely a natural-language string: it comprises a terminal goal predicate $G(s_T)$, trace constraints $J(\tau)$, authority boundaries, and evidence requirements.

Long horizon is multidimensional. Sequence length matters, but so do dependency depth, branching, delayed reuse of earlier artifacts, interacting constraints, state hidden outside the current context, and the distance between a fault and the observation that reveals it. We call the resulting quantity *effective task horizon*. It is an empirical construct to be operationalized by a task dependency graph rather than equated with prompt length or token count.

2.2 Characteristic failures

The architecture targets five failure classes.

1. **Intent drift**: a local objective remains plausible but no longer refines the approved goal.
2. **Authority drift**: a machine makes a scientific, release, or external decision that required human authority.
3. **Unsupported completion**: an artifact is marked complete without evidence satisfying its acceptance predicate.
4. **Stale dependency propagation**: a repaired upstream object leaves downstream artifacts and evidence incorrectly trusted.
5. **context fracture**: a fresh session lacks the assumptions, decisions, or evidence needed to continue safely.

These failures can occur even when every generated sentence is locally coherent. They require state, contracts, evidence, and invalidation semantics, not only stronger prose prompting.

3 Hierarchical Execution Model

3.1 Levels and roles

The canonical levels are shown in [Table 1](#).

Level	Lifetime	Semantic responsibility	Normal authority
Goal	programme	global outcome, constraints, authority	human + Main
Plan	several phases	strategic DAG and checkpoints	human approval, Main ownership
Phase	milestone	coherent machine-managed sub-goal	Main/Theory/Experiment
DIC	phase/repair-stable	durable intent, acceptance, evidence, escalation	owning workstream
EPS	disposable	just-in-time instantiation of planned prompt/run nodes	owning workstream
Prompt/Run	attempt-local	bounded agent execution and context	owning workstream/subagent
Action	primitive	tool use, edit, search, experiment, test	executor/tool

Table 1: The hierarchy separates durable semantics from disposable operations.

Main owns overall authority, architecture, integration, state, and publication. Theory owns formal reasoning, literature and novelty analysis, propositions, and counterexamples. Experiment owns benchmark design, implementation, statistics, and empirical evidence. Search and research are capabilities available to Theory and Experiment. A workstream may delegate bounded actions, but it retains its DIC and integration responsibility.

3.2 Plans and phases

Definition 3.1 (Plan). *A Plan is a tuple $P = (V, E, \prec, \Gamma, h)$ where (V, E) is a directed acyclic graph of phases, $\prec$ is a declared execution order compatible with E , Γ is the governance policy, and h is the periodic human-checkpoint interval. Each phase $v \in V$ has an owner, objective, dependency set, materiality class, DIC pointer, status, and evidence index.*

A Plan is the normal human approval object. A Phase is a machine milestone, not ordinarily a separate human approval request. The runtime permits automatic progression to a dependency-ready ordinary phase until a checkpoint becomes due or a material escalation opens.

3.3 Durable intent contracts

Definition 3.2 (DIC). *A durable intent contract for phase v is*

$$C_v = (D_v, I_v, A_v, W_v, \mathcal{U}_v, \mathcal{X}_v, \mathcal{R}_v),$$

where $D_v \subseteq \mathcal{S}$ is a precondition domain, $I_v \subseteq \mathcal{T}$ is a trace invariant, $A_v \subseteq \mathcal{S}$ is an acceptable post-state set, $W_v : \mathcal{S} \times \mathcal{E}^* \rightarrow \{0, 1\}$ is an evidence predicate, $\mathcal{U}_v$ is the authority/input set, $\mathcal{X}_v$ is the escalation set, and $\mathcal{R}_v$ is the declared dependency/read footprint.

The runtime materializes one DIC package per Phase rather than a sequence of stepwise DICs. Required views are `README.md` (general summary), `REQUESTS.md` (specific goals and acceptance requirements), and `PLAN.md` (the durable execution graph, including prompt identifiers, dependencies, parallel subagent batches, merge points, and conditional repair/review branches). Optional `ARCHITECTURE.md`, `TEST_MATRIX.md`, and user-defined flat Markdown views expose additional angles on the same contract. A machine `MANIFEST.json` indexes these views and content hashes. A DIC survives local repair. If the subgoal, acceptance meaning, frozen protocol, or authority changes materially, the correct operation is replan or escalate rather than silently mutating the contract.

3.4 EPS, prompt/runs, and actions

The DIC execution plan is durable, while an EPS is a disposable just-in-time instantiation of selected plan nodes against the current repository state, model/context, attempt, and repair history. An EPS may name prompt/run IDs, dependencies, parallel groups, subagent delegations, expected outputs, local checks, and stop conditions, and it may materialize executable prompt files. Ordinary failure or retry therefore creates a new EPS/run instance without changing the DIC unless the Phase contract or durable execution design itself changes. An Action record links to an EPS and, when applicable, a prompt/run node, while preserving DIC, Phase, Plan, result, and evidence lineage.

4 Intent and Contract Refinement

4.1 Trace semantics

Associate each object X in the hierarchy with an admitted trace set $\llbracket X \rrbracket \subseteq \mathcal{T}_X$. Adjacent levels may use different state descriptions, so let $\rho_{XY} : \mathcal{T}_X \rightarrow \mathcal{T}_Y$ be a declared abstraction map.

Definition 4.1 (Semantic refinement). *X refines Y , written $X \preceq_\rho Y$, if*

$$\rho_{XY}(\llbracket X \rrbracket) \subseteq \llbracket Y \rrbracket.$$

This definition makes intent preservation a semantic inclusion claim, not a similarity judgment between prompt strings. In a practical package, refinement is supported by contract predicates, evidence, tests, and review; it is rarely decided by a complete semantic oracle.

Proposition 4.2 (Refinement-chain preservation). *If $X_0 \preceq X_1 \preceq \dots \preceq X_m$ and the abstraction maps compose on admitted traces, then $X_0 \preceq X_m$.*

Proof. Repeated image inclusion gives $\rho_{0m}(\llbracket X_0 \rrbracket) \subseteq \rho_{1m}(\llbracket X_1 \rrbracket) \subseteq \llbracket X_m \rrbracket$. $\square$

The proposition is a consequence of the definition. It does not show that a model-generated child actually refines its parent; that requires a sound certificate or verifier.

4.2 Acceptance predicates and acceptable-state sets

For a completed phase trace τ_v ending in state s' , define

$$\text{Acc}_{C_v}(\tau_v, e_v) = \mathbf{1}\{s' \in A_v, \tau_v \in I_v, W_v(s', e_v) = 1\}.$$

Acceptance separates the desired state from evidence that warrants believing the state was reached. A unit test may be strong evidence for a deterministic software property and weak evidence for a broad scientific claim. The DIC must state the connection.

4.3 Compositional satisfaction

For each phase v , let F_v be its implemented transition relation. Let $\text{Pred}(v)$ be its predecessors. Write Pre_v and Post_v for its pre/post predicates and Frame_v for keys it promises not to alter.

Assumption 4.3 (Contract soundness). *For every phase v , every admitted execution satisfies the Hoare-style relation $\{\text{Pre}_v \wedge I\} F_v \{\text{Post}_v \wedge I\}$, and its evidence predicate is sound for the asserted postcondition.*

Assumption 4.4 (Edge compatibility). *For each non-source v , the conjunction of relevant predecessor postconditions implies Pre_v . Concurrent or unordered branches either have disjoint writes or satisfy an explicit commutativity/noninterference condition.*

Theorem 4.5 (Compositional satisfaction). *For an acyclic Plan satisfying contract soundness and edge compatibility, if the initial state satisfies all source preconditions and the conjunction of terminal postconditions implies the global goal G , then every admitted complete topological execution terminates in a state satisfying G and the global invariant I .*

Proof. Proceed by induction over a topological order. Source preconditions hold initially. At step v , all predecessors have executed; edge compatibility and noninterference imply Pre_v , while the induction hypothesis gives I . Contract soundness yields Post_v and preserves I . After all nodes, all terminal postconditions and I hold; the terminal implication gives G . $\square$

The assumptions are substantial. Hidden dependencies, unsound evidence, stochastic tools outside the contract, or incompatible branches invalidate the conclusion. The theorem is a precise restatement of contract composition in this setting, not a claim that LLM agents automatically satisfy it (Hoare, 1969).

5 Verification and Replanning

5.1 Closed-loop operator

Verification consumes the DIC, current state, emitted evidence, and relevant dependency versions. It returns

$$\mathcal{V}(C_v, s, e) \in \{\text{accept, repair, replan, escalate}\}.$$

Accept closes the DIC. **Repair** retains the DIC and regenerates EPS. **Replan** changes the decomposition or contract and therefore crosses a governance boundary. **Escalate** requests authority unavailable to the active workstream.

Local verification is automatic where a tool or predicate can check the action. Independent review is reserved for material evidence, claim freeze, release, or submission boundaries. Independence means a fresh context and no authorship of the audited artifact; merely repeating the same prompt does not remove shared blind spots.

5.2 Imperfect verification

Let a fault remain present at successive gates. Define M_i as the event that it survives gate i undetected.

Assumption 5.1 (Conditional miss bound). *For some $0 \leq \alpha < 1$, $\Pr(M_i \mid M_1 \cap \dots \cap M_{i-1}) \leq \alpha$ at every gate at which the fault is observable.*

Theorem 5.2 (Survival and detection bound). *Under the conditional miss bound,*

$$\Pr(M_1 \cap \dots \cap M_k) \leq \alpha^k.$$

If N is the number of gate intervals through which the fault persists before detection, then $\mathbb{E}[N] \leq (1 - \alpha)^{-1}$.

Proof. The chain rule and the conditional bounds give the product bound. The tail-sum formula gives $\mathbb{E}[N] = \sum_{k \geq 0} \Pr(N > k) \leq \sum_{k \geq 0} \alpha^k = (1 - \alpha)^{-1}$. $\square$

No independence assumption is needed beyond the stated conditional bound. The bound is useless when the fault is not represented in the evidence channel, a limitation formalized in [Definition 23.1](#).

5.3 Repair versus replan

A repair changes operational means while preserving the DIC’s semantics and authority. A replan changes at least one parent-level obligation, dependency, acceptance meaning, or frozen protocol. This distinction is auditable: a repair produces a new EPS under the same DIC hash; a replan changes a DIC or Plan revision and opens a checkpoint or escalation event.

6 Dependency Locality and Failure Propagation

6.1 Declared footprints

Model state as a key-value map $s : \mathcal{K} \rightarrow \mathcal{V}$. Each phase v declares read set $R_v \subseteq \mathcal{K}$ and write set $W_v \subseteq \mathcal{K}$. Edges include every possible data dependence: if $W_u \cap R_v \neq \emptyset$, then $u \rightarrow v$ unless an equivalent versioned external dependency is declared.

After a repair changes keys $\Delta \subseteq \mathcal{K}$, define the direct-reader seed

$$B(\Delta) = \{v : R_v \cap \Delta \neq \emptyset\}$$

and dynamic impact cone

$$\mathcal{I}(\Delta) = \text{Reach}_G(B(\Delta)),$$

including the seed.

Assumption 6.1 (Local deterministic-or-reverified semantics). *A phase’s output and evidence validity depend only on its declared read values, declared predecessor outputs, explicit tool/version parameters, and fresh randomness recorded in evidence. There is no undeclared mutable shared state. Evidence is bound to input and artifact versions.*

Theorem 6.2 (Dependency-local recovery). *Under complete dependency edges and local deterministic or reverified semantics, after a repair that changes only Δ , retaining completed phase outputs and evidence for every $v \notin \mathcal{I}(\Delta)$ preserves their contract validity. Re-execution need be considered only inside $\mathcal{I}(\Delta)$.*

Proof. Take a topological order of nodes outside the cone. A node $v \notin \mathcal{I}(\Delta)$ does not directly read a changed key. Nor can any predecessor output on which it depends have changed: otherwise a changed predecessor would reach v , placing v in the cone. By induction, all declared inputs, tool/version parameters, and predecessor outputs of v are unchanged. Local semantics therefore preserve its output distribution under recorded randomness, or fresh reverification certifies it, and version-bound evidence remains valid. $\square$

Theorem 6.3 (Worst-case tightness). *Without additional semantic restrictions, every node in $\mathcal{I}(\Delta)$ can be necessary to invalidate: for any such node, there exists a set of phase functions consistent with the declared graph for which changing Δ changes that node’s accepted output. Hence no strict subset of the impact cone is uniformly safe over the declared-footprint model.*

Proof. For a direct reader, choose its output to copy a changed bit. Along any path from that reader to a target descendant, choose each phase to copy the predecessor bit and let its acceptance predicate bind to the previous version. The change propagates to the target and invalidates its old evidence. Thus each reachable node is necessary in some admissible realization. $\square$

This result explains both the value and fragility of local repair. Complete dependency declarations can reduce rework from a full rerun to $\sum_{v \in \mathcal{I}(\Delta)} c_v$; one hidden edge can make that guarantee false. The structure parallels dependency graphs, incremental build systems, and self-adjusting computation (Ferrante et al., 1987; Mokhov et al., 2018; Acar, 2009).

6.2 Propagation bounds

A fault originating at node u can affect at most $|\text{Desc}(u)| + 1$ phases and in the worst case affects all of them. If the descendant subgraph has branching at most b and depth at most d , then

$$|\text{Desc}(u)| + 1 \leq \sum_{i=0}^d b^i,$$

with the usual $d + 1$ limit when $b = 1$. Verification gates do not alter this structural cone; they reduce how much of it executes before detection and which outputs remain trusted.

7 Context and Persistent State

7.1 Package as authority

Chat history is not a durable database. The package therefore stores a small authoritative index and immutable or versioned semantic records. A fresh workstream reads the approved goal, Plan, active Phase, complete multi-view DIC, EPS/prompt/action lineage, compact reports surface, evidence index, checkpoints, and escalations from the package. Generated narrative handoffs are useful views, not competing authorities.

7.2 Context projection

Let $\Pi_v(s)$ be the context projection supplied to the executor for phase v , and let $\pi_v(a \mid s)$ be the ideal phase action kernel.

Proposition 7.1 (Context-local execution). *Omitting state outside $\Pi_v(s)$ leaves phase behavior unchanged if*

$$\pi_v(a \mid s) = \pi_v(a \mid \Pi_v(s))$$

for all admissible s, a , all acceptance predicates and escalation conditions are measurable from the projection plus resolvable evidence references, and omitted state cannot mutate through a hidden channel. If two admissible states share a projection but require different action distributions or acceptance decisions, the projection is insufficient.

Proof. The sufficient direction follows by substitution in the action and verification kernels. For necessity, take states s, s' with $\Pi_v(s) = \Pi_v(s')$ but different required behavior; any projection-only executor must act identically on them and is wrong on at least one. $\square$

Externalized state reduces context load only when the projection is semantically sufficient. It is not made safe by storing more files whose relevance cannot be recovered. MemGPT and hierarchical memory systems motivate externalization and working-memory control (Packer et al., 2023; Wang et al., 2024); the DIC specifies what must be projected for one bounded phase.

8 Human Oversight Across Timescales

8.1 Governance complexity

Suppose a Plan has N ordinary completed phases, periodic checkpoint interval h , and M material escalations. Beyond initial Plan approval, the number of mandatory human governance events is at most

$$\left\lfloor \frac{N}{h} \right\rfloor + M$$

(up to a chosen final release review). This is a counting invariant of the policy, not a quality guarantee. Machine-local checks may be much more frequent.

8.2 Checkpoint interval under imperfect verification

Consider a simplified stationary model. Each checkpoint costs c_v ; a false rejection occurs with probability β and costs c_f ; faults arrive at rate λ per phase; re-executing one contaminated phase costs c_r ; a persisting observable fault is missed at a gate with conditional probability $\alpha < 1$. With interval h , a fault appears uniformly within its first block. Its expected contaminated span before detection is

$$\frac{h}{2} + h \frac{\alpha}{1 - \alpha} = h \frac{1 + \alpha}{2(1 - \alpha)}.$$

Ignoring overlapping faults and edge effects, expected overhead over n phases is

$$U(h) = \frac{n(c_v + \beta c_f)}{h} + \lambda n c_r h \frac{1 + \alpha}{2(1 - \alpha)}.$$

Proposition 8.1 (Cost-optimal interval in the stationary model). *The continuous minimizer is*

$$h^* = \sqrt{\frac{2(c_v + \beta c_f)(1 - \alpha)}{\lambda c_r (1 + \alpha)}}.$$

The discrete policy uses a feasible integer near h^ .*

Proof. U is strictly convex for $h > 0$. Setting its derivative to zero and solving gives the expression. $\square$

The formula recovers the qualitative checkpoint/restart trade-off (Young, 1974; Daly, 2006): expensive review favors longer intervals; frequent or costly faults favor shorter intervals; a weak verifier changes both detection delay and repair overhead. The runtime default of seven is a configurable operating prior, not an estimate of h^* for every project.

9 Main, Theory, and Experiment Specialization

Specialization is semantic, not ceremonial. Main resolves authority conflicts and maintains the Plan. Theory is expected to search primary sources, define domains, attempt counterexamples, and downgrade claims that do not survive. Experiment freezes protocols, implements objective graders, records complete evidence, and separates smoke, pilot, confirmatory, and post-hoc analyses. Each can refine the durable DIC execution plan, generate EPS run instances, and delegate concrete prompt/runs or actions, so a persistent Orchestrator is unnecessary.

The separation reduces one common failure: the same context invents a claim, designs a favorable test, interprets the result, and approves publication. It does not guarantee independence. Material theory and evidence should receive a fresh hostile review at a release boundary.

10 Reference Implementation and Repository Architecture

The reference implementation is not merely illustrative pseudocode. Version 5.6.35 ships a repository-resident runtime, schemas, templates, validation rules, migration support, tests, and an already-instantiated example Plan. This section specifies those surfaces in enough detail that a fresh Main, Theory, or Experiment session can reconstruct how the package is intended to operate without relying on the conversation that created it. The guiding rule is that durable scientific and governance state belongs in the package, whereas transient model context is disposable.

10.1 Authority hierarchy and source-of-truth policy

The package distinguishes semantic authority from operational convenience. The highest durable machine-readable index is `.ai-workflow/STATE.json`. It identifies the active Plan, Plan approval state, Phase order and dependencies, role ownership, DIC locations and hashes, EPS instances, action records, evidence pointers, checkpoint counters, escalation events, and a bounded event history. It is intentionally an index rather than a replacement for richer Phase-local artifacts.

For an active Phase, the normal authority order is:

1. the approved human Goal and Plan recorded in package state;
2. the active Phase record in `STATE.json`;
3. the Phase's single multi-view DIC/ package;
4. frozen scientific or experimental authorities explicitly referenced by that DIC;
5. the current EPS instance and prompt/run materializations;
6. logs, evidence, tool outputs, and compact Phase reports.

An EPS may refine how work is performed but may not silently change a request, acceptance condition, authority boundary, frozen protocol, or material scientific claim. If such a change is required, the DIC, Phase, or Plan must be revised at the appropriate level and the corresponding governance event recorded.

10.2 Canonical package layout

New long-horizon work uses the hierarchy under `.ai-workflow/runtime/plans/`. A representative package is:

```
.ai-workflow/
|-- STATE.json
|-- MODEL_ROUTING.json
|-- config.toml
|-- aw/
|-- docs/
|-- schema/
```

```

|-- tests/
'-- runtime/
    '-- plans/
        '-- <plan-id>/
            '-- phases/
                '-- <phase-id>/
                    |-- DIC/
                    |   |-- README.md
                    |   |-- REQUESTS.md
                    |   |-- PLAN.md
                    |   |-- ARCHITECTURE.md      # optional / auto
                    |   |-- TEST_MATRIX.md       # optional / auto
                    |   |-- <CUSTOM_VIEW>.md     # zero or more
                    |   '-- MANIFEST.json
                    |-- EPS/
                    |   |-- <eps-id>.json
                    |   '-- prompts/
                    |       '-- <eps-id>__<node>.md
                    |-- actions/
                    |-- logs/
                    |-- evidence/
                    |-- minor/                  # off | auto | on
                    |-- research/              # off | auto | on
                    |-- verification/          # off | auto | on
                    |-- checkpoints/           # off | auto | on
                    '-- reports/               # mandatory, flat

```

Substantive project outputs are not automatically copied into the Phase directory. Source code remains in source directories, manuscripts remain under **submissions/**, research ledgers remain under **research/**, and experiment outputs remain under the experiment tree. The Phase directory stores the control contract, execution lineage, and compact evidence surface that explain why those artifacts exist and whether they are accepted.

10.3 Plan records

A Plan is the normal human governance object. It spans several Phases and contains at least a title, global Goal, approval state, Phase order, dependency graph, checkpoint interval, active Phase identifier, and completed-since-checkpoint counter. The default checkpoint interval is seven completed ordinary Phases; values around five to ten are expected for long research programmes, but the interval is configurable per Plan.

Plan approval is not equivalent to approval of every future prompt. It authorizes ordinary machine progression inside the declared objectives, dependencies, and escalation policy. A material objective change, authority change, scientific-claim change, frozen-protocol change, release/submission change, purchase or paid-compute action, credential/private-system access, or destructive/external action outside authority triggers immediate escalation regardless of checkpoint cadence.

10.4 Phase records and lifecycle fields

A Phase is a coherent machine-managed milestone, not a chat session and not a single prompt. It may involve tens or hundreds of prompt/runs. The Phase record in **STATE.json** stores the compact index needed for dependency management and recovery, including:

- identifier, title, objective, owner, and materiality;
- dependencies and lifecycle status;
- DIC identifier, path, hash, and registered views;
- EPS identifiers and paths;
- Action identifiers and paths;

- acceptance predicates and their status;
- verification outcomes and evidence references;
- lane policy and report policy;
- attempt, creation, start, and completion timestamps.

The canonical status progression is `pending` → `active` → `complete`, with blocked or escalated states when dependencies or governance prevent progression. Completing a Phase can automatically make the next dependency-ready Phase eligible to start; a human is not asked to approve that ordinary transition.

11 The Multi-View Durable Intent Contract

The v5.6.35 DIC is deliberately richer than a single JSON object. There is normally one DIC package for the whole Phase. Its files describe the same Phase from complementary angles. They are not DIC-01/DIC-02/DIC-03 execution steps. This distinction is important for phases that require 10–100+ prompt/runs: the Phase objective should remain durable while the operational route evolves.

11.1 DIC manifest and package semantics

`DIC/MANIFEST.json` is the machine index for the multi-view package. It stores the DIC identifier, Plan/Phase linkage, owner, objective, constraints, authority, acceptance predicates, evidence requirements, out-of-scope items, escalation triggers, dependencies, registered views, and whether custom views are allowed. The manifest does not replace the Markdown views. Instead it allows validation and recovery to determine which views exist and which are required.

The DIC hash protects against accidental semantic drift. Ordinary EPS repair should not modify durable DIC content simply to make a failing run easier to complete. A DIC change is justified when the Phase contract itself changes: for example, a theorem target is weakened after falsification, an experiment estimand is redefined, a repository migration changes compatibility promises, or a new venue rule changes the required publication artifact.

11.2 README.md: orientation and recovery view

`README.md` is the concise orientation surface. It explains what the Phase is, why it exists, its owner and authority boundaries, the principal unresolved questions, the important source artifacts, and the recommended read order for a fresh session. It should not duplicate every request or every prompt node. Its function is to restore situational awareness quickly.

A strong DIC README normally includes:

- Phase purpose and relation to the approved Plan;
- current scientific/implementation state;
- frozen facts and non-negotiable constraints;
- important input paths;
- DIC view map;
- brief completion criterion;
- escalation boundaries.

11.3 REQUESTS.md: specific goals and acceptance inventory

`REQUESTS.md` is the authoritative goal inventory for the Phase. A Request states what must be achieved or resolved, not which prompt executes first. Requests can therefore be satisfied by one prompt, many prompts, or a later discovery that renders an earlier planned branch unnecessary. Useful states include `OPEN`, `PARTIAL`, `SATISFIED`, `BLOCKED`, `SUPERSEDED`, and `REMOVED`.

A Request should be specific enough to admit evidence-based closure. For example:

R03 -- Dependency-local recovery

Goal: determine the strongest correct locality result supported by the runtime.

Acceptance:

- formal statement or explicit obstruction;
- assumptions and failure domain stated;
- proof/counterexample retained;
- hostile audit performed if theorem-level claim survives;
- claim ledger synchronized.

Phase completion is governed by Request resolution and acceptance evidence, not by blind exhaustion of the prompt list. A Phase with 80 completed prompt/runs can remain incomplete if a required Request is unresolved; conversely, a decisive result may satisfy several Requests and make planned nodes unnecessary.

11.4 PLAN.md: durable execution graph

The DIC Plan is explicitly operational. It records which prompt/run nodes are intended, their dependencies, which may execute in parallel, which are delegated to subagents, which outputs must be merged, where milestone reviews occur, and which conditional branches are activated after failure, falsification, or successful theorem/evidence survival. Stable prompt IDs are coordination references rather than human approval boundaries.

A typical fragment is:

```
Batch A -- recover and map
P01  recover authoritative state
P02A positive derivation          \
P02B hostile counterexample       > parallel subagent batch
P02C nearest-literature search   /
P03  reconcile P02A/P02B/P02C

Conditional branch
if P03 yields theorem-level claim:
  P04 independent audit
else:
  P04R obstruction characterization
```

The DIC Plan is durable because these relationships can remain meaningful across retries. If P02A fails because an assumption was overlooked, the system can instantiate another run of P02A or insert P02A-R without redefining the Phase's overall objective. If the discovery materially changes the durable route—for example, the entire positive-theorem branch is removed and the Phase becomes an impossibility study—then the DIC Plan should be revised and the change recorded.

11.5 Parallelism and subagent declarations

Parallelism is explicit rather than inferred after the fact. A Plan should separate genuinely independent branches from merely convenient concurrency. Positive derivation, hostile attack, and literature search are often parallelizable because they should not contaminate one another. By contrast, a synthesis prompt depends on the outputs it integrates and must not start before those outputs are available.

For each parallel node, the Plan may specify:

- Request IDs served;
- dependency predecessors;
- parallel group;
- preferred workstream/model profile;
- subagent status;
- permitted write scope;

- expected output/evidence;
- integration consumer.

Subagents are bounded executors. They do not inherit Plan authority merely because they were delegated a task. A subagent may investigate, edit, calculate, search, or run experiments inside the declared contract; Main, Theory, or Experiment remains responsible for integration and material decisions.

11.6 ARCHITECTURE.md: target repository/system architecture

ARCHITECTURE.md is optional or automatically materialized when a Phase changes repository, runtime, module, schema, CLI, interface, or other system architecture. It should describe current and target architecture, interfaces, ownership, migration rules, compatibility guarantees, invariants, and important failure boundaries. It is not a generic theory document.

A repository-reconstruction Phase might specify files to add/remove, directory migrations, command aliases, schema version changes, protected compatibility surfaces, and state-recovery requirements. A Phase that does not change architecture should normally omit the view rather than create a ceremonial placeholder.

11.7 TEST_MATRIX.md: concrete test or experiment matrix

TEST_MATRIX.md records combinations that must actually be tested, compared, falsified, or audited. It is intentionally broader than a simple acceptance checklist. Depending on Phase type it can encode software test batches, algorithm $\times$ dataset $\times$ seed matrices, condition $\times$ horizon $\times$ fault matrices, theorem candidate $\times$ derivation $\times$ hostile audit matrices, or manuscript $\times$ venue-rule $\times$ build checks.

For example:

Batch	Conditions	Horizon	Fault	Seeds	Primary outcome
A	C0/C1/C2/C3	4	none	10	end-to-end success
B	C2/C3	12	injected	10	recovery locality
C	C1+V/C2-extra/C3	12	injected	10	matched-budget mechanism contrast

The matrix belongs in the DIC when it is part of the durable Phase design. Individual run instantiations, random seeds selected from an already-frozen set, or retry-specific commands belong in EPS/log/evidence surfaces.

11.8 Custom DIC views

Users may add arbitrary flat Markdown views and register them in the DIC manifest. The runtime intentionally avoids a closed ontology because different research phases need different durable perspectives. Useful examples include:

- THEOREM_MAP.md: candidate theorem/lemma graph, dependencies, status, and required audits;
- EXPERIMENT_PROTOCOL.md: frozen models, seeds, stopping rules, exclusions, and estimands;
- DATA_CONTRACT.md: provenance, schemas, leakage restrictions, and transformation rules;
- GRADER_SPEC.md: objective scoring behavior and edge cases;
- NOVELTY_TARGETS.md: nearest-neighbour claims requiring primary-source comparison;
- VENUE_REQUIREMENTS.md: page limits, anonymity, required statements, and mutable dead-lines;
- MIGRATION.md: old-to-new schema/API mapping;
- RELEASE_CONTRACT.md: version, compatibility, artifact, and packaging requirements.

A custom view is not a sub-Phase. It is another authoritative angle on the same DIC. DIC views remain flat to keep recovery and validation simple.

12 Execution Prompt Stack and Prompt/Run Lifecycle

12.1 DIC Plan versus EPS

The DIC Plan and EPS both concern execution, but their lifetimes differ. The DIC Plan states the durable graph—for example, “P11A and P11B are parallel; P12 depends on both; P13 exists only if the theorem survives.” The EPS instantiates those nodes against the current repository state, model context, attempt history, available tools, and newly discovered constraints.

An EPS is therefore disposable. It may be superseded without altering the DIC when the semantic contract is unchanged. This is the principal mechanism for allowing a long Phase to adapt autonomously while preserving intent.

12.2 EPS representation

The reference EPS schema stores Plan, Phase, and DIC lineage; executor; creation time; node list; stop conditions; and, per node, operation, local acceptance, stable Plan node identifier, dependencies, parallel group, subagent flag, model profile where available, status, and materialized prompt path. The `EPS/prompts/` directory stores the complete runnable prompt/run text so that execution can be reproduced or audited even after the live model context is gone.

A future high-resolution Phase may have many EPS objects. For example, one Phase can begin with an EPS covering P01–P25, create a repaired EPS for P11 after a failed proof, materialize a fresh independent-review EPS after a theorem survives, and later create another EPS for manuscript integration. The DIC remains one package.

12.3 Prompt/run materialization

A prompt/run is a bounded execution unit. A materialized prompt should normally contain:

1. desired outcome and the Request/Plan node it serves;
2. exact authoritative context or file read order;
3. hard constraints and prohibited changes;
4. evidence and local success criteria;
5. allowed write/tool scope;
6. autonomy and escalation boundary;
7. required deliverable and record location.

The prompt should not force visible chain-of-thought. It should request decisions, evidence, artifacts, and concise rationale. The model is allowed to choose the micro-path unless the path itself is part of the DIC contract.

Stable prompt IDs are important even though humans do not approve each prompt. They permit dependency tracking, logs, supersession, failure analysis, subagent routing, and exact resumption. Prompt numbering is machine coordination, not governance ceremony.

12.4 Bounded subagent contract

A delegated subagent should receive enough context to complete its bounded objective but not the authority to reinterpret the whole programme. A practical contract contains:

- node ID and objective;
- authoritative input paths;
- immutable facts;
- allowed files/directories and tools;
- forbidden material changes;
- expected output/evidence schema;
- failure and stop conditions;
- return format to the owning workstream.

For independent hostile review, fresh context is preferable and the reviewer should not have authored the audited artifact. Independence is an execution property, not merely a different role label in the same contaminated context.

12.5 Parallel execution and integration

Parallelism can reduce wall time and increase epistemic diversity, but only when integration is explicit. Every parallel group should have a later consumer node that reconciles contradictions and records which branch is authoritative. Raw branch outputs should remain available in logs/evidence even if the integrated conclusion is short.

The workflow distinguishes parallelism for efficiency from parallelism for independence. Two implementation workers may share context when performing separable edits. A positive-theorem derivation and hostile counterexample search should preferably use independent contexts so that one does not anchor the other. A release verifier should use a fresh context at a higher materiality boundary.

12.6 Retries, repair nodes, and supersession

Ordinary failure is handled inside the same DIC when possible. A failed prompt can be retried, repaired, or superseded. The Plan/EPS lineage should record why the replacement exists and which downstream nodes depend on the repaired output. Repeating an identical failing prompt without changing information or assumptions is not a valid repair strategy.

A typical repair sequence is:

```
P11A    positive derivation
  -> fails: hidden-state assumption discovered
P11R1   characterize the missing assumption
P11R2   rederive with explicit assumption
P11V    local or independent audit
P12     reconcile with negative and literature branches
```

If the repair changes the Phase's actual target rather than merely its route, the durable DIC must be revised.

13 Phase Execution Lanes, Evidence, and Reports

13.1 Core execution surfaces

Every canonical Phase materializes DIC/, EPS/prompts/, actions/, logs/, evidence/, and reports/. Actions are primitive execution/provenance records. Logs preserve detailed run history. Evidence stores phase-local evidence and indices into authoritative project artifacts. These surfaces may be large; they are not the primary human review interface.

13.2 Optional lanes and off|auto|on

The lanes `minor/`, `research/`, `verification/`, and `checkpoints/` use a three-state policy:

off the lane is prohibited for the Phase;

auto the default; the lane is absent until first needed, then materialized lazily;

on the lane is created and maintained from Phase initialization.

This policy preserves useful machinery without forcing every Phase to begin with a large empty tree. It also lets a frozen experiment explicitly disable Minor changes when apparently small edits could contaminate the protocol.

13.3 Minor lane

Minor is a bounded local-maintenance lane for work such as stale links, headings, filenames, index synchronization, small documentation repairs, and deterministic expectation updates. It must be promoted out of Minor when a request changes architecture, public interfaces/dependencies, scientific method or claims, frozen experiment protocol, data policy, release/submission state, external authority, credentials, or destructive behavior.

Minor can run in parallel with primary Phase work when its files do not conflict. It should not become a shadow backlog for substantive work. In a long Phase the benefit of the lane is isolation: dozens of small cleanup requests can be processed without bloating the main prompt graph or forcing the human to approve each item.

13.4 Research lane without a persistent Researcher role

Research is a capability/lane rather than a required persistent role. Theory uses it for primary literature and novelty; Experiment uses it for benchmarks, methods, data, and implementation evidence. The lane can hold query records, source matrices, literature notes, downloaded evidence metadata, and novelty comparisons. Research findings do not become implementation or scientific evidence merely because they are well cited; the owning workstream must integrate them into a decision, claim, protocol, or artifact.

13.5 Verification lane

Verification uses three practical intensities:

1. **local**: deterministic tests, compilation, schema checks, evidence existence, re-reading a derivation, or a same-workstream sanity check;
2. **milestone**: a lightweight second-model/fresh-context check after a meaningful prompt cluster or before closing an important Request;
3. **independent**: fresh-context hostile review at theorem, frozen-evidence, release, or submission boundaries.

The runtime's semantic outcomes are **accept**, **repair**, **replan**, and **escalate**. Independent review is reserved for material boundaries rather than mechanically invoked after every prompt. This is a deliberate correction to overly granular verifier ceremony.

13.6 Checkpoint lane

Human checkpoints are Plan-level governance events that may have Phase-local supporting material. They normally occur after several completed Phases, not after every Phase. The checkpoint surface should summarize what changed since the last review, which Requests or claims changed status, unresolved risks, current evidence strength, resource use where material, and the proposed next several Phases. An overdue checkpoint prevents continued autonomous progression until resolved, unless the runtime policy explicitly permits otherwise.

13.7 Mandatory compact reports/ surface

Every Phase has a **reports/** directory. Unlike the optional lanes, reports cannot be disabled. The directory is deliberately constrained:

1. no subdirectories;
2. maximum ten files by default;
3. must include `REPORT.md`;
4. must include key merged raw results, with `RAW_RESULTS.jsonl` as the default surface;
5. must include `MANIFEST.json`;
6. full forensic material remains elsewhere.

The purpose is to create a compact canonical Phase output surface that both humans and fresh agents can inspect quickly. A Phase should usually use three to seven report files, not fill the quota merely because ten are allowed.

13.8 Merged-raw semantics

“Merged raw” means lossless collation plus provenance. A merged raw file may concatenate records, attach source identifiers/paths/hashes, normalize text encoding, or normalize a schema/column order when the operation is lossless. It must not silently average, round, paraphrase, drop failed cases, select favourable examples, or rewrite a model result into a narrative. For experiments, each seed/run can remain a row. For Theory, raw derivation/audit outputs can remain discrete records. For software, raw test/build results can be collated verbatim with exit status and source path.

This creates a two-tier evidence model:

full forensic evidence in `logs/`, `evidence/`, and lanes $\longrightarrow$ compact merged raw in `reports/`
 $\longrightarrow$ concise synthesis in `REPORT.md`.

The concise report may interpret; the raw report surfaces may not silently transform the underlying result.

14 Operational Execution Protocol

This section gives the end-to-end protocol expected for a substantial research programme. It is intentionally more detailed than the one-page Quick Start because the technical report is also a recovery and implementation reference.

14.1 Step 1: initialize and recover package state

A fresh Main should first establish that the package is the intended repository and run the recovery/status path. It reads `STATE.json`, active Plan metadata, open escalations/checkpoints, and the active or next-ready Phase. If recovery detects missing DIC views, broken hashes, impossible dependencies, malformed report surfaces, or inconsistent lineage, those defects are repaired before scientific execution continues.

The canonical hierarchical command surface is exposed through:

```
python .ai-workflow/aw_hierarchy.py status
python .ai-workflow/aw_hierarchy.py recover
```

The older `aw` CLI remains available for v5.6.31-compatible Goal/Structured repositories, but it is subordinate to the hierarchical authority for new work.

14.2 Step 2: create the Plan and Phase graph

The Plan should decompose the human programme into a small number of coherent Phases with explicit dependencies and owners. Phases should be large enough that human approval after each would be unnecessary; a Phase can contain 10–100+ prompts. Conversely, a Phase should not be so broad that it has no coherent acceptance state or cannot be owned by one workstream.

Example command pattern:

```
python .ai-workflow/aw_hierarchy.py plan-create \
  "Research programme" \
  "Approved global objective" \
  --checkpoint-interval 7

python .ai-workflow/aw_hierarchy.py phase-add \
```

```

"Formal theory" \
"Establish or falsify the candidate theory" \
--owner Theory

python .ai-workflow/aw_hierarchy.py phase-add \
"Confirmatory experiment" \
"Run the frozen controlled evaluation" \
--owner Experiment \
--depends phase-formal-theory

```

The identifiers returned by the runtime are used in later commands. The human approves the Plan once the objective, Phase graph, authority, and checkpoint policy are acceptable.

14.3 Step 3: approve the Plan

Approval establishes the normal machine-autonomy boundary. It authorizes ordinary work inside the Plan and DICs; it does not authorize paid compute, credentials, external submissions, destructive external operations, or a material objective/claim/protocol change unless those actions were explicitly included in authority.

```
python .ai-workflow/aw_hierarchy.py plan-approve --by human
```

14.4 Step 4: construct the Phase DIC package

Before a substantial Phase begins, the owning workstream ensures the DIC is sufficiently complete. The three core views are always README.md, REQUESTS.md, and PLAN.md. Architecture and test/experiment matrix views are added when relevant. Custom views are encouraged when they reduce ambiguity rather than duplicate prose.

The DIC Plan should normally be written at the prompt-node level before or during early execution. It need not predict every repair node in advance, but it should identify the intended batches, dependencies, parallel branches, merge points, and major decision branches. For an overnight run this graph is especially important because it allows the system to continue through dozens of executions without asking the human for procedural instructions.

14.5 Step 5: configure lanes and report policy

Default Phase lanes are `auto`. A Theory Phase may set `research=on`; an Experiment Phase may set `minor=off` after protocol freeze; a release Phase may set `verification=on`. Examples:

```

python .ai-workflow/aw_hierarchy.py lane-set <phase> research on
python .ai-workflow/aw_hierarchy.py lane-set <phase> minor off
python .ai-workflow/aw_hierarchy.py lane-set <phase> verification auto

```

A lane in `auto` state is materialized only when first needed. Populated lanes are not silently deleted merely because configuration changes.

14.6 Step 6: start the Phase and recover its read set

Starting a Phase requires an approved Plan and completed dependencies. The owning workstream then reads the DIC core views and only the source evidence required for the first batch. Context is selected by dependency and authority, not by indiscriminately loading the entire repository.

```

python .ai-workflow/aw_hierarchy.py phase-start <phase-id>
python .ai-workflow/aw_hierarchy.py prompt Theory <phase-id>

```

The generated role prompt is a concise execution brief, not the entire DIC. The DIC and source files remain the detailed authority.

14.7 Step 7: instantiate the next EPS batch

The active workstream converts the relevant DIC Plan nodes into an EPS against current state. The EPS may refine exact file read sets, model choice, tools, attempt-specific assumptions, write scopes, and local acceptance checks. Parallel nodes may be instantiated together.

The CLI accepts a structured JSON node list. Conceptually:

```
python .ai-workflow/aw_hierarchy.py eps-generate <phase-id> '[
  {"id": "P11A", "operation": "positive derivation",
    "depends_on": [], "parallel_group": "B1", "subagent": true},
  {"id": "P11B", "operation": "hostile counterexample search",
    "depends_on": [], "parallel_group": "B1", "subagent": true},
  {"id": "P12", "operation": "reconcile branches",
    "depends_on": ["P11A", "P11B"]}
]
```

Exact fields are schema-governed; the durable semantic relationship is that EPS nodes instantiate DIC Plan nodes rather than inventing unrelated work.

14.8 Step 8: execute prompts and preserve raw evidence

Each prompt/run performs bounded work and records outputs. Deterministic tool outputs, experiment rows, build logs, citations, and artifact hashes are preserved rather than paraphrased away. Large raw outputs can remain outside **reports/**; the report surface receives only the key merged raw subset plus provenance.

The owner integrates subagent outputs rather than accepting them independently. Conflicting outputs are retained and explicitly reconciled. A model's self-report that a test passed is not evidence when the test command can be run directly.

14.9 Step 9: local verification throughout execution

Local checks occur as close as possible to the action that can fail. Code is tested; LaTeX is compiled; schemas are validated; generated files are reopened; experiment outputs are checked for expected counts and missingness; mathematical statements are dimension/domain checked; citations are inspected against primary sources when load-bearing. Local verification is automatic and does not require a separate persistent Verifier session.

14.10 Step 10: milestone review after meaningful clusters

After several substantive nodes or before accepting a major Request, the owner may invoke a lightweight fresh-context/model recheck. The purpose is to detect drift before it contaminates a large downstream cone. This is particularly useful in long overnight runs where a human will not review every intermediate result.

Milestone review should be proportionate. Mechanical documentation changes do not need frontier independent review; a theorem-level result or protocol-freeze decision does.

14.11 Step 11: interpret verification outcomes

The runtime records four outcomes:

accept evidence satisfies the relevant acceptance conditions;

repair the DIC remains valid but one or more artifacts/runs need bounded correction;

replan the durable execution design or decomposition is no longer adequate;

escalate the issue crosses a human/material authority boundary.

repair normally creates new EPS/runs under the same DIC. **replan** may modify DIC/PLAN.md, dependencies, or Phase decomposition and should record why. **escalate** stops autonomous progression on the affected scope until resolved.

14.12 Step 12: handle faults by dependency-local repair

When a fault changes state keys or authoritative artifacts, downstream validity is determined by declared dependencies rather than by blanket restart. A local repair can preserve independent branches. The theoretical conditions for safe locality are formalized in [Section 6](#); operationally, the package must record enough read/write/evidence relationships to know what might be stale.

If hidden dependencies are suspected, the repair cone is conservatively widened. Locality is a conditional optimization, not an excuse to ignore uncertain coupling.

14.13 Step 13: maintain Requests and reports during the Phase

Request status is updated as evidence accumulates. The Phase report remains concise and may be regenerated from current state, but raw merged records are append-preserving. Important negative results, failed tests, and falsified branches belong in the report/evidence surface when they constrain future work.

The ten-file report cap forces aggregation by semantic role instead of endless report proliferation. For example, fifty per-run experiment JSON files should remain in the raw run archive while `RAW_RUNS.jsonl` collates the relevant records in `reports/`.

14.14 Step 14: independent review at material boundaries

Fresh independent review is appropriate before accepting a theorem-level claim, freezing confirmatory evidence, releasing a package, or submitting a manuscript. The reviewer should receive the relevant DIC, raw evidence, and target artifact but should not inherit the author's private reasoning history. The reviewer returns an evidence-linked verdict, not a rewritten project narrative.

14.15 Step 15: Phase closure

A Phase closes only when required Requests and acceptance conditions are resolved and the report/evidence surfaces are valid. Prompt exhaustion alone is insufficient. Closure records final evidence, completion time, verification outcome, and lineage. The next dependency-ready Phase can proceed automatically unless a checkpoint or escalation is due.

```
python .ai-workflow/aw_hierarchy.py phase-complete <phase-id> \
  --evidence reports/REPORT.md \
  --evidence reports/RAW_RESULTS.jsonl
```

14.16 Step 16: periodic human checkpoint

After the configured number of ordinary completed Phases, Main presents a compact checkpoint rather than every underlying prompt. The checkpoint should answer:

- which Phases completed and which Requests materially changed;
- strongest new evidence/claims and their confidence/evidence class;
- negative or blocked results;
- deviations from the approved Plan;
- current resource or deadline constraints;
- proposed next several Phases;
- any decision only the human can make.

The checkpoint may resolve to `continue`, `replan`, or `stop`. Material escalations do not wait for the periodic checkpoint.

15 Role Specialization, Routing, and Execution Intensity

15.1 Main

Main owns programme authority, architecture, Plan decomposition, cross-workstream integration, durable state, and publication. Main can orchestrate its own Phase; a separate persistent Orchestrator is unnecessary. Main should resist absorbing every raw research transcript or test log into its live context. Instead it reads the DIC, compact reports, and decisive evidence required for the current decision.

15.2 Theory

Theory owns formal reasoning, conceptual analysis, literature/novelty, propositions, counterexamples, and claim classification. Its DIC Plan should often separate positive derivation, hostile negative search, primary-literature search, and reconciliation into explicit branches. Useful custom views include `THEOREM_MAP.md`, `CLAIM_POLICY.md`, and `NOVELTY_TARGETS.md`. A failed proof attempt is normally an EPS event; it becomes a DIC change only if the durable scientific target changes.

15.3 Experiment

Experiment owns empirical methodology, task/benchmark design, implementation delegation, frozen protocols, statistics, and evidence. Useful DIC views include `TEST_MATRIX.md`, `EXPERIMENT_PROTOCOL.md`, `DATA_CONTRACT.md`, and `GRADER_SPEC.md`. After protocol freeze, apparently minor changes can be scientifically material, so `minor=off` may be appropriate. Experiment reports should expose key raw run/grader/metric surfaces rather than only aggregates.

15.4 Model routing

The shipped quality-first profile assigns GPT-5.6 Pro as the default for Main, Theory, and Experiment, with independent frontier review when a material evidence, claim-freeze, release, or submission boundary is reached. The package also retains compatibility routing guides for ChatGPT-only, Claude-only, and mixed surfaces. These labels are preferences, not claims that a provider or model is available at runtime.

The general routing rule is: reserve high-capability, high-context models for semantic decomposition, theorem/evidence adjudication, architecture, integration, and publication; delegate bounded deterministic or mechanical work to cheaper capable models when checks make the delegation safe. The cheapest model is not automatically optimal if repeated repair costs exceed the initial saving.

15.5 Execution intensity and proportionality

Legacy low/mid/high/xhigh/ultra guides remain useful as risk/intensity profiles even though they are not additional persistent roles. A trivial deterministic edit should not invoke the same ceremony as a locked scientific release. Conversely, a destructive migration, private-data action, expensive external experiment, or final submission requires stronger authority, rollback, immutable run identity, and independent verification.

The operative principle is proportionality: increase decomposition, independent review, evidence retention, and rollback requirements with coupling, irreversibility, scientific materiality, and cost—not merely with prompt length.

16 Verification, Replanning, and Human Governance in Operation

16.1 Immediate material escalation categories

The reference runtime recognizes explicit material categories: objective change, authority change, scientific-claim change, protocol change, destructive/external action, release change, submission change, purchase or paid compute, and credentials/private-system access. The list is intentionally redundant with human instructions because material boundaries should be machine-checkable and visible in state.

16.2 What does not normally require escalation

Ordinary prompt retries, tool failures, local code repairs, literature dead ends, theorem attempts that fail without changing the publication objective, reordering independent prompt nodes, adding a bounded subagent, or regenerating an EPS from the same DIC are machine-execution decisions. Treating all of these as human gates would recreate the phase/prompt ceremony the hierarchy was designed to remove.

16.3 Repair versus replan

A repair corrects execution while preserving the durable contract. Examples include fixing code to satisfy an unchanged API, re-running a statistic after a parsing error, replacing a flawed proof step while keeping the theorem statement and assumptions, or regenerating a prompt with the newly discovered context it should have read.

A replan changes durable execution design but can remain inside the same Phase if objectives and authority are unchanged. Examples include replacing a planned positive proof route with an impossibility route, adding a new experiment control necessary for interpretable attribution, or restructuring a repository migration after a compatibility constraint is discovered. If the objective, acceptance, frozen protocol, or authority changes materially, replan may require human escalation.

16.4 Checkpoint semantics

Checkpoint cadence is independent of prompt count. A Phase with 100 prompt/runs still counts as one completed Phase for the default periodic review unless the Plan explicitly defines another governance rule. This preserves the user's intended attention model: approve the programme, allow many machine-level executions, and review batches of substantive milestones.

17 State Recovery, Context Construction, and Handoffs

17.1 Fresh-session recovery order

A new session should be able to reconstruct the programme from the package alone. The recommended read order is:

1. `.ai-workflow/STATE.json` or hierarchical `status`;
2. active Plan and open checkpoint/escalation state;
3. active Phase DIC `README.md`, `REQUESTS.md`, `PLAN.md`;
4. relevant optional DIC views;
5. `reports/REPORT.md` plus raw report surfaces;
6. source evidence specifically required by the next Plan nodes;
7. current EPS instance and prompt/run files if resuming an interrupted attempt.

This order avoids two extremes: relying on a tiny status summary with insufficient semantics, and loading the entire repository into context.

17.2 Context projection policy

Context is a projection of durable state chosen for a bounded decision. A prompt should include the smallest sufficient read set that preserves correctness. Large source files, old logs, and unrelated Phases remain externalized until needed. The correctness condition is the conditional sufficiency proposition stated earlier: omitted history is safe only if it cannot change the action distribution or acceptance decision conditional on the retained projection.

17.3 Handoff quality

A handoff is good when the next session can identify the current objective, authoritative files, completed evidence, failed attempts that matter, open Requests, active EPS node, and next decision without reconstructing the original conversation. The package should not require a bespoke narrative handoff after every prompt because DIC, reports, and state already provide that function. Narrative reports are reserved for compact synthesis and decision support.

18 CLI and Configuration Reference

18.1 Hierarchical runtime commands

The canonical command surface is intentionally small. The principal commands are:

Command	Purpose
<code>init</code>	initialize hierarchical state and checkpoint default
<code>status</code>	show active Plan, Phases, checkpoints, and escalations
<code>recover</code>	audit package-only reconstructability and lineage
<code>migrate</code>	index/migrate reasonable v5.6.31 state
<code>plan-create</code>	create the human-scale Plan
<code>phase-add</code>	add a Phase with owner, dependencies, and acceptance
<code>plan-approve</code>	record the human Plan approval boundary
<code>dic-create</code>	materialize the Phase's multi-view DIC package
<code>dic-view-add</code>	register a custom flat DIC view
<code>eps-generate</code>	instantiate Plan prompt nodes for current state
<code>lane-set</code>	set minor/research/verification/checkpoints to off/auto/on
<code>lane-materialize</code>	explicitly create an enabled optional lane
<code>phase-start</code>	activate a dependency-ready Phase
<code>phase-complete</code>	close an accepted Phase with evidence
<code>verify</code>	record accept/repair/replan/escalate outcome
<code>escalate</code>	create an immediate material escalation
<code>checkpoint-resolve</code>	continue, replan, or stop at a checkpoint
<code>lineage</code>	reconstruct Phase execution lineage
<code>prompt</code>	emit a role-specific concise execution brief

The CLI performs deterministic state and file operations; it is not itself an LLM Orchestrator.

18.2 Core configuration

Repository defaults expose DIC view and lane policy:

```
[phase_lanes]
minor = "auto"
research = "auto"
verification = "auto"
```

```
checkpoints = "auto"

[dict_views]
readme = "on"
requests = "on"
plan = "on"
architecture = "auto"
test_matrix = "auto"
allow_custom = true

[reports]
policy = "compact_raw"
flat = true
max_files = 10
include_key_raw_results = true
```

README, REQUESTS, and PLAN are core DIC views. ARCHITECTURE and TEST_MATRIX are optional/automatic. Reports are not toggleable. Per-Plan/Phase state is authoritative over repository defaults when the two differ.

18.3 Compatibility CLI

The older `aw` command family still exposes Goal/Structured phase, request, prompt, Minor, model, and root-management functions. It exists for v5.6.31-style repositories and migration. New long-horizon programmes should not maintain a legacy `phases/phase_XX/` tree as a competing authority beside the hierarchical runtime. Compatibility commands should either be treated as wrappers or clearly isolated legacy state.

19 Migration and Backward Compatibility

Version 5.6.35 preserves useful v5.6.31 artifacts rather than deleting them. Legacy Goal/context/handoff files can be indexed. Orchestrator maps to Main, and Researcher maps to Theory by default or Experiment when the historical task is empirical. Older prompt/request/minor records remain evidence of past work but do not supersede the canonical Plan/Phase/DIC state.

The migration from v5.6.32 single-file DICs into multi-view DIC packages is conservative. Existing nine completed Phases were reconstructed into README, REQUESTS, PLAN, manifests, EPS prompts, and report surfaces. Where v5.6.32 had not recorded the true detailed 10–100 prompt graph, the migrated Plan explicitly says that the historical graph is reconstructed from coarse records. The migration does not fabricate unrecorded prompts merely to make history look more complete.

20 Conformance, Validation, and Failure Policy

20.1 Engineering conformance coverage

The runtime tests cover at least: one Plan authorizing several Phases; autonomous ordinary progression; periodic checkpoint triggering; immediate material escalation; reconstructable Plan–Phase–DIC–EPS–Prompt/Run–Action lineage; Main/Theory/Experiment ownership; package-only state recovery; compatibility with reasonable v5.6.31 usage; dependency blocking; verification outcomes; multi-view DIC creation; custom DIC views; EPS prompt materialization; optional lane semantics; and mandatory flat report-surface validation. These tests establish control-plane behavior, not language-agent task superiority.

20.2 Report validation

Validation rejects a missing **reports/** directory, nested report directories, more files than the configured cap, or absence of the required concise/raw/manifest surfaces. This rule is intentionally strict because report proliferation was a source of ambiguity in earlier workflow versions.

20.3 Failure retention

Failed runs, falsified theorems, negative experiments, build errors, and rejected verifier outputs should be retained when they constrain future interpretation. The workflow must not present only the final successful trajectory. Raw evidence, attempt IDs, and supersession relationships make failure informative rather than embarrassing state to be erased.

20.4 Resource and external-action policy

The workflow may plan expensive experiments or external actions, but it may not silently cross authority boundaries. Paid compute, purchases, credentials/private systems, external submissions, and destructive actions outside the working package require explicit authority. Resource qualification should precede scale: confirm that the code path, memory/VRAM/storage assumptions, grader, and logging work on a small instance before launching expensive batches.

21 Worked Phase Profiles

21.1 Theory Phase profile

A substantial Theory Phase may use:

```
DIC/
  README.md
  REQUESTS.md
  PLAN.md
  THEOREM_MAP.md
  TEST_MATRIX.md
EPS/
logs/
evidence/
research/          on or auto
verification/      auto
reports/
  REPORT.md
  RAW_RESULTS.jsonl
  CLAIM_STATUS.json
  AUDIT_RESULTS.jsonl
  LITERATURE_MATRIX.csv
  MANIFEST.json
```

The Plan may define independent positive derivation, counterexample, and novelty branches; a merge node; a conditional fresh theorem audit; and manuscript reconciliation. A theorem can be downgraded without asking the human if the global publication objective survives; a change from “theory-supported mechanism paper” to a materially different objective should escalate.

21.2 Experiment Phase profile

A confirmatory Experiment Phase may use:

```
DIC/
  README.md
  REQUESTS.md
  PLAN.md
```

```

TEST_MATRIX.md
EXPERIMENT_PROTOCOL.md
GRADER_SPEC.md
EPS/
logs/
evidence/
minor/           off after protocol freeze
research/        auto
verification/    auto or on
reports/
  REPORT.md
  RAW_RUNS.jsonl
  RAW_GRADER_RESULTS.jsonl
  RAW_METRICS.csv
  PROTOCOL.json
  ANALYSIS.csv
  MANIFEST.json

```

The Plan states condition batches, dependency structure, parallel seeds/runs, objective graders, integration/statistical analysis, and failure branches. Protocol changes after freeze are material and must not be smuggled through Minor or EPS repair.

21.3 Runtime reconstruction Phase profile

A repository/runtime Phase commonly materializes `ARCHITECTURE.md` and `TEST_MATRIX.md`. Its Plan can run independent implementation branches in parallel only where write scopes do not collide, followed by integration, unit tests, migration tests, recovery audit, and release validation. Reports preserve raw test/build results alongside the concise architecture decision.

21.4 Publication Phase profile

A submission Phase can add custom DIC views such as `VENUE_REQUIREMENTS.md`, `PAPER_OUTLINE.md`, or `RELEASE_CONTRACT.md`. The Plan should separate source-rule verification, manuscript drafting, citation verification, build/render inspection, anonymity checks, and final independent review. External submission itself is outside the Phase’s autonomous authority unless the human explicitly authorizes it.

22 Detailed End-to-End Example

Consider a Theory Phase whose Request is to determine whether dependency-local recovery supports a nontrivial theorem and whether it is novel enough to motivate experiments. The DIC Plan can schedule P01 to recover formal objects; P02A to attempt a positive theorem; P02B to search hidden-dependency counterexamples; P02C to inspect build-system/incremental-computation literature; and P03 to reconcile them. P02A–C run in parallel under separate contexts. P03 determines that a theorem survives only under complete dependency declarations. It adds P04A for a tightness construction and P04B for a fresh hostile proof audit. The audit finds the theorem correct but not independently novel. P05 updates the claim ledger and report, while P06 revises the empirical hypothesis from “hierarchy is novel” to “the integrated verified-local-repair mechanism may improve long-horizon recovery.”

Nothing in this sequence requires human approval if the approved objective was to establish the strongest defensible theory and conservative novelty position. The EPS changes several times; the DIC Requests remain stable. If the result instead implied that the entire intended submission theme was invalid, Main would record a scientific-claim/objective escalation and ask the human whether to reframe the programme.

This example illustrates the intended granularity. The Phase is one coherent milestone; the DIC is one durable multi-view package; the Plan contains many prompt nodes; EPS objects instantiate current attempts; subagents run bounded branches; reports merge key raw results; and human attention is reserved for programme-level changes.

23 Formal Properties and Limits

23.1 What is guaranteed by construction

Identifier lineage, role normalization, checkpoint counters, escalation categories, atomic state replacement, and DIC hash checks are implementation invariants when callers use the canonical runtime. They are not semantic guarantees about the world.

23.2 Observability limit

Let $O : \mathcal{S} \rightarrow \mathcal{O}$ be the evidence observation available to a verifier and $G : \mathcal{S} \rightarrow \{0, 1\}$ the relevant goal predicate.

Proposition 23.1 (Evidence-aliasing impossibility). *If there exist s_+, s_- such that $O(s_+) = O(s_-)$ but $G(s_+) = 1$ and $G(s_-) = 0$, then no verifier $V : \mathcal{O} \rightarrow \{\text{accept}, \text{reject}\}$ can have both zero false acceptance and zero false rejection on $\{s_+, s_-\}$.*

Proof. V receives the same input on both states and must return the same decision. Accepting falsely accepts s_- ; rejecting falsely rejects s_+ . $\square$

No amount of hierarchical decomposition overcomes evidence that fails to distinguish success from failure. The remedy is a richer environment-grounded observation, a weaker claim, or explicit residual uncertainty—not a more confident verifier prompt.

23.3 Open-world and adversarial limits

The theory assumes declared dependencies, stable identifiers, and evidence tied to the asserted property. Open-world side effects, prompt injection, mutable external services, and adversarial evidence can violate those assumptions. ToolEmu and AgentDojo illustrate why sandbox and security evaluation are separate requirements (Ruan et al., 2024; DeBenedetti et al., 2024). The runtime is a control layer, not a security boundary.

23.4 Novelty limit

The formal ingredients have established analogues in program logics, runtime verification, build systems, and checkpointing (Hoare, 1969; Leucker and Schallhart, 2009; Mokhov et al., 2018; Daly, 2006). The report therefore claims neither theorem novelty in isolation nor implementation equivalence to a learned hierarchical world model. LeCun’s world-model programme and joint-embedding predictive models are conceptual motivation for abstraction across timescales, not descriptions of this symbolic package protocol (LeCun, 2022; Bardes et al., 2024).

24 Empirical Hypotheses and Evaluation Design

24.1 Workshop-stage objective and evidence standard

The immediate evaluation is deliberately smaller than the full research programme. The near-term objective is a workshop-quality proof of concept under a fixed practical resource envelope: one ChatGPT Plus account supplies persistent Main sessions, while two Claude Pro accounts

supply bounded Claude Code execution. The experiment asks a focused mechanism question: *on a small set of evolving, stateful, and scientific terminal tasks, does the full package-resident workflow preserve accumulated requirements and recover more locally than a competent flat ChatGPT–Claude Code workflow using the same basic model-account stack?*

This is not intended to estimate population-level benchmark performance or establish a universal superiority claim. Five task families and fourteen scored trajectories can expose mechanism-relevant failures, generate inspectable checkpoint curves, and determine whether a larger study is scientifically warranted. The package therefore forbids headline significance testing from this study and requires task-level and checkpoint-level reporting. The broader factorial C0–C3/horizon/fault protocol is preserved separately in `experiments/FULL_RESEARCH_PROTOCOL_FUTURE.md`; it is future work rather than an implied result of the workshop experiment.

24.2 Model/account architecture and contamination boundary

ChatGPT is Main in every scored run. Each run starts from a fresh Main chat to prevent one benchmark trajectory from teaching Main the solution to a later trajectory; Main remains persistent within the run so that the hierarchy is exercised as intended. Claude Code authenticated through Claude Pro A is the primary bounded terminal executor for all ten core runs. Claude Pro B is reserved for four pre-declared replications—Flat and Workflow on `database_migration` and `recli`—plus exceptional fresh-context review if a release-critical artifact is ambiguous. Actual surfaced model identifiers, client versions, and account roles are recorded in the run evidence; the protocol does not silently infer a model version from a subscription label.

The workflow must not gain an unrecorded resource advantage. Both scored conditions may plan, inspect files, execute tools, run tests, and iteratively debug. The difference is organizational state and recovery semantics, not whether one condition is artificially prevented from behaving competently. Paid cloud environments, paid API inference, or additional accounts are outside the authorized workshop protocol and trigger a resource escalation before use.

24.3 Primary comparison: Flat versus Workflow

The study uses two primary conditions rather than a large ablation matrix.

F—Flat competent workflow. ChatGPT receives the task, maintains an ordinary task-level plan, delegates bounded work to Claude Code, reads results, requests corrections where useful, and may run normal tests or inspect the environment. It does *not* materialize a Phase DIC, persistent Request ledger, durable prompt/dependency graph, EPS lineage, evidence-gated four-outcome state machine, or dependency-local invalidation structure. Flat is therefore a strong plan-and-execute comparator rather than a one-shot straw man.

W—AI-Native Workflow. ChatGPT Main reconstructs the approved package state and operates the full Phase/DIC/EPS protocol. A task Phase has one multi-view DIC; `REQUESTS.md` preserves accumulated requirements, `PLAN.md` records prompt/run dependencies, parallel sub-agents and merge/conditional nodes where justified, and `TEST_MATRIX.md` is used when the task admits an explicit matrix. Planned nodes are instantiated just in time through EPS and bounded Claude Code runs. Evidence is retained and verification resolves to accept, repair, replan, or escalate. When a fault is localized, Main attempts to reopen only the justified dependency cone rather than reflexively restarting all prior work.

Both conditions receive the same benchmark task information. Oracle solutions and hidden verifier internals are excluded from scored contexts. A successful W run may use fewer or more executor invocations than F; invocation count and wall time are outcomes rather than forced equalities. If later work seeks a causal matched-inference-budget claim, that requires an additional controlled experiment.

ID	Task	Source	Mechanism role
T1	<code>env_manager</code>	SlopCodeBench	easier evolving-specification control; tests whether hierarchy merely adds overhead
T2	<code>database_migration</code>	SlopCodeBench	accumulated dependencies, rollback, and preservation of earlier requirements
T3	<code>recli</code>	SlopCodeBench	harder eight-checkpoint evolving CLI; strongest small-budget long-horizon proxy
T4	<code>live-database-cutover</code>	Terminal-Bench	live state and cutover correctness; stresses evidence before consequential transitions
T5	<code>certified-sparse-regression</code>	Terminal-Bench-Science	scientific-transfer/certification case outside ordinary repository engineering

Table 2: Frozen workshop-stage task families. The exact Harbor task revision/image/verifier hashes are resolved and recorded before scored runs.

24.4 Five-task frozen workshop set

The task set is intentionally small but heterogeneous. Three tasks come from SlopCodeBench, whose multi-step Harbor formulation progressively reveals requirements while earlier requirements remain active, directly exposing regression under accumulated specification (Orlanski et al., 2026). One current Terminal-Bench task supplies a live stateful systems problem (Merrill et al., 2026), and one Terminal-Bench-Science task supplies scientific/certification transfer (Terminal-Bench Science Team, 2026). Exact task identifiers and resolved versions are locked by Phase 02 before scored execution.

The benchmark names do not themselves guarantee suitability. Phase 01 prepares the environments without scored execution, and Phase 02 must run non-scored availability/oracle/verifier qualification, inspect task resources, and freeze exact identifiers. A verifier defect, unavailable required service, or unauthorised paid infrastructure can block or replace a task only through a documented pre-score protocol decision.

24.5 Fourteen-run schedule

The core experiment consists of one F/W pair for each of the five tasks using Claude Pro A, for ten trajectories. Four additional trajectories replicate the two most mechanism-relevant SlopCodeBench tasks with Claude Pro B: T2-F, T2-W, T3-F, and T3-W. The run order is predeclared in `experiments/workshop_poc/RUN_MATRIX.csv` and alternates condition order across the core pairs to reduce simple order effects. No trajectory is silently rerun because its score is disappointing; infrastructure-invalid runs must be retained with a reason and replacement rule.

The resulting sample is too small for asymptotic claims. Replication on T2/T3 is used as a robustness check on the most important qualitative pattern, not as a claim that two accounts constitute independent model families.

24.6 Outcomes

The primary external outcome is the official benchmark reward/pass result. For SlopCodeBench, strict checkpoint correctness and regression/erosion across checkpoints are central because the mechanism claim concerns preservation under cumulative requirements. The package also records: (i) rework or repeated work, (ii) repair scope—the amount of previously accepted work reopened after a detected failure, (iii) Claude Code executor-run count, (iv) Main interaction/decision count, and (v) wall time. Raw grader outputs, transcripts or transcript pointers, artifact hashes, client versions, termination reasons, and environment locks are retained.

The analysis is descriptive and paired. Every task is shown separately. For multi-step tasks, checkpoint curves are preferred to a single endpoint whenever available. Aggregate win counts or mean rewards may be shown only alongside the underlying task rows. With five task families, the workshop paper does not report a p-value as evidence of general superiority.

24.7 Mechanism-oriented expectations

Three patterns are informative even without a large sample. First, W may be neutral or costlier on the easier T1 control; this is consistent with a fixed coordination cost. Second, any relative advantage should be more visible on T2/T3 if persistent requests and explicit recovery matter as specification depth accumulates. Third, T4 can expose whether verification is connected to consequential state transitions rather than merely producing critiques. T5 asks whether the control protocol remains usable in a scientific/certification task.

These are hypotheses, not promised outcomes. A null result in which F and W perform similarly, or an adverse result in which W introduces unproductive ceremony, must be retained and used to narrow the claimed scope.

24.8 Execution phases and evidence freeze

The approved evaluation Plan separates engineering from scoring. Phase 01 is repository engineering, CLI cleanup, GitHub publication under explicit credential authority, official-template integration, and benchmark-environment preparation; it must not execute scored F/W runs. Phase 02 qualifies and freezes environments. Phase 03 executes the ten primary paired runs with Claude Pro A. Phase 04 executes the four T2/T3 replications with Claude Pro B. Phase 05 reconciles raw evidence and claims. Phase 06 inserts only verified results into the workshop manuscripts and rebuilds submission artifacts.

This separation prevents implementation debugging from being confused with experimental evidence. The frozen protocol, task catalogue, run matrix, condition specification, model-budget record, environment-preparation guide, run schema, and evidence/analysis rules are package-resident under `experiments/workshop_poc/`.

24.9 Current evidence boundary

At version 5.6.35, no scored workshop trajectory has been run. Existing workflow tests and dependency-local fixtures are engineering/conformance evidence only. The workshop drafts therefore contain methods, formal claims, and explicit result-insertion boundaries but no invented comparative numbers. Empirical claims become admissible only after Phases 03–05 generate and reconcile the declared evidence.

25 Related Work

Planning and search. ReAct, Tree of Thoughts, Graph of Thoughts, ReWOO, ADaPT, and LATS develop reasoning/action interleaving, explicit search, or adaptive decomposition (Yao et al., 2023b,a; Besta et al., 2024; Xu et al., 2023; Prasad et al., 2024; Zhou et al., 2024a). AI-Native Workflow operates at a longer-lived authority and artifact layer; it can host such methods inside EPS.

Compound agents. CAMEL, AutoGen, AgentVerse, and MetaGPT establish role-based multi-agent coordination (Li et al., 2023; Wu et al., 2023; Chen et al., 2024; Hong et al., 2024). The present focus is durable refinement, evidence, invalidation, and governance rather than the novelty of manager/worker roles.

Memory and context. MemGPT and HiAgent address external or hierarchical memory (Packer et al., 2023; Wang et al., 2024). The context-locality proposition states the sufficiency condition needed for safe omission; it does not propose a new neural memory model.

Benchmarks. AgentBench, WebArena, GAIA, SWE-bench, OSWorld, and τ -bench expose stateful, tool-mediated, and long-horizon failures (Liu et al., 2024; Zhou et al., 2024b; Mialon et al., 2024; Jimenez et al., 2024; Xie et al., 2024; Yao et al., 2024). The immediate workshop study additionally uses Terminal-Bench for difficult command-line tasks (Merrill et al., 2026), SlopCodeBench for cumulative-specification regression (Orlanski et al., 2026), and Terminal-Bench-Science for scientific workflow transfer (Terminal-Bench Science Team, 2026). This five-task proof of concept is deliberately narrower than the controlled horizon/fault generator retained for a future full study.

Verification and oversight. Reflexion, process supervision, debate, recursive oversight, and weak-to-strong supervision motivate feedback and scalable checking (Shinn et al., 2023; Lightman et al., 2023; Irving et al., 2018; Leike et al., 2018; Burns et al., 2024). The four-outcome operator adds explicit repair/replan/escalation semantics, while the observability proposition prevents treating verifier confidence as ground truth.

Harness optimization. DSPy and automated design of agentic systems optimize prompts or agent graphs (Khattab et al., 2024; Hu et al., 2025). AI-Native Workflow specifies how a chosen harness is authorized, executed, audited, and locally recovered. Direct comparative evidence is future work.

26 Limitations

The architecture increases explicit state and may add overhead on short tasks. DIC quality is itself model- and human-dependent. Read/write footprints can be incomplete; hidden dependencies invalidate local recovery. Evidence can be stale, adversarial, or weakly connected to the claim. Verifier errors may be correlated, violating convenient rate models. The checkpoint optimum assumes stationarity and ignores overlapping faults, queueing, and heterogeneous costs. Role specialization does not create epistemic independence. The package has no confirmatory evidence yet that the proposed hierarchy improves end-to-end agent performance. Workshop templates and deadlines are external mutable constraints and must be rechecked immediately before submission.

27 Conclusion

AI-Native Workflow 5.6.35 refines phase-by-phase prompt ceremony with a simpler hierarchy and clearer authority: a human approves a Plan; Main, Theory, and Experiment execute bounded phases through durable contracts and disposable operations; evidence-gated verification chooses accept, repair, replan, or escalate; and declared dependencies make local recovery auditable. The formal analysis identifies exactly what composition and locality require, quantifies one imperfect-verification trade-off, and proves that evidence aliasing prevents universal correctness. The next scientific step is the package-frozen five-task, fourteen-run workshop proof of concept: compare a competent Flat ChatGPT–Claude Code workflow with the full package-resident protocol, preserve task/checkpoint-level evidence, and determine whether the harder evolving tasks exhibit less regression or more local recovery. The larger matched-budget horizon/fault programme is retained as future work rather than presupposed by this release.

A Claim classification summary

ID	Classification	Status
ARCH-01/02	invariant by construction	implemented and test-covered
DEF-REF-01	definition	retained
THM-COMP-01	theorem under assumptions	proved; not novelty claim
THM-LOCAL-01/02	theorems under assumptions	proved; locality and tightness
THM-VER-01	theorem under assumptions	proved; conditional miss bound
PROP-CHK-01	proposition under assumptions	proved; stationary model
PROP-CTX-01	proposition under assumptions	proved; conditional sufficiency
IMP-OBS-01	impossibility proposition	proved
HYP-HORIZON-01	empirical hypothesis	future full-study hypothesis; not tested by five-task PoC
HYP-REPAIR-01	empirical hypothesis	targeted descriptively in workshop PoC; no result yet

B Required experiment record

For the workshop proof of concept, each run record must conform to `experiments/workshop_poc/schema/RUN_R` and include package Git commit, task/resolved benchmark version, condition, ChatGPT Main surface, Claude Code account/model/client identifiers, environment/tool versions, transcript pointer or preserved transcript, artifact hashes, grader/verifier outputs, checkpoint outcomes where applicable, rework/repair-scope records, wall time, executor/Main counts, and termination/infrastructure-validity status. The future full study additionally records task seeds, controlled dependency graphs and injected-fault metadata. Aggregate manuscript tables must be generated from these records rather than hand-copied.

C Ordinary execution brief

A modern prompt needs the desired outcome, package/domain authority, hard constraints, evidence and success criteria, autonomy/escalation boundary, and deliverable. The model chooses the route and records artifacts and concise rationale; it is not asked to reveal private chain-of-thought.

D Operational Checklists

The following checklists are normative operating aids. They are not all mandatory for every low-risk task; Main applies them proportionately to the Phase’s materiality. Their purpose is to

prevent a long autonomous run from reaching the end with an avoidable authority, evidence, or state defect.

D.1 Before starting a Phase

1. Confirm that the intended Plan is approved and no unresolved material escalation blocks execution.
2. Confirm that all declared Phase dependencies are complete and their reports/evidence remain valid.
3. Read the DIC README, Requests, and Plan; inspect Architecture/Test Matrix/custom views when present.
4. Confirm that each required Request has an acceptance interpretation and that no Request silently contradicts a frozen upstream decision.
5. Confirm that DIC/PLAN.md is detailed enough for the anticipated scale. A Phase expected to require 50 prompts should not begin with a three-line “analyze, implement, report” plan unless early mapping is itself the first declared batch.
6. Confirm owner: Main, Theory, or Experiment.
7. Confirm lane policy; explicitly disable Minor after protocol freeze where needed.
8. Confirm reports exists and is flat; record the maximum file cap.
9. Identify any external, destructive, credential, purchase, or submission action that requires separate authority.
10. Run package recovery/validation before relying on the state as authoritative.

D.2 Before delegating a subagent prompt

1. Give the stable Plan node ID and Request IDs served.
2. State exact outcome and local acceptance, not merely “help with this.”
3. Provide authoritative inputs and immutable facts.
4. Bound write scope and tool permissions.
5. State what the subagent must not reinterpret: Goal, protocol, public API, theorem statement, etc.
6. Specify raw output/evidence retention.
7. Specify return path and integration owner.
8. For an epistemically independent branch, use fresh context and avoid leaking the competing branch’s conclusion.
9. For parallel writes, ensure branches do not mutate the same authoritative artifact without an explicit merge policy.
10. Record actual model/surface when material to reproducibility.

D.3 Before accepting a theory claim

1. Define domains, dimensions, quantifiers, and assumptions explicitly.
2. Separate definition/invariant from theorem/proposition and from empirical hypothesis.
3. Attempt at least one hostile counterexample route for a material theorem.
4. Check hidden state/dependencies when the claim concerns locality, modularity, or composition.
5. Verify proof steps independently when the result is publication-load-bearing.
6. Search nearest primary literature before claiming novelty.
7. Record known analogues and state whether novelty is theorem-level, mechanism-level, integration-level, or only empirical.
8. Synchronize the claim ledger, report, and manuscripts.
9. Preserve failed stronger statements rather than silently deleting their history if they constrain interpretation.

D.4 Before freezing an experiment

1. Fix the primary question, estimand, task families, conditions, and primary outcomes.
2. Freeze model identifiers/versions where possible, prompts, seeds or seed-generation procedure, budgets, stopping rules, exclusions, and grader version.
3. Confirm controls distinguish the proposed mechanism from extra calls/tokens, stronger prompts, or generic critique.
4. Validate data/task leakage protections.
5. Validate fault injector invisibility and reproducibility.
6. Run smoke/pilot work only for engineering/variance planning; do not reuse pilot choices as unreported confirmatory tuning.
7. Decide whether `minor=off` is required after freeze.
8. Ensure raw run and grader formats include package hash and provenance.
9. Record the protocol in a durable DIC view and compact report surface.

D.5 Before a destructive or external action

1. Confirm the action is within explicit human authority.
2. Identify target system, files, repository, branch, account, or service exactly.
3. Determine whether credentials/private systems are involved.
4. Determine whether the action incurs cost or paid compute.
5. Establish rollback or backup when feasible before mutation.
6. Prefer dry-run/read-only inspection first.
7. Record what evidence will demonstrate successful completion.
8. If the action is a submission/release, separate artifact readiness from the final external click/API action.

D.6 Before closing a Phase

1. Reconcile every required Request; do not equate prompt completion with Phase completion.
2. Run relevant local tests/compilation/schema checks.
3. Ensure any material claim/evidence boundary has received appropriate independent review.
4. Confirm `reports/REPORT.md`, `RAW_RESULTS.jsonl`, and `MANIFEST.json` exist.
5. Confirm reports is flat and within the file cap.
6. Ensure negative/failed results that constrain the next Phase are visible.
7. Ensure key project outputs are referenced by path/hash and are not stale relative to repaired dependencies.
8. Update evidence pointers and final verification outcome.
9. Run lineage/recovery validation.
10. Check whether closure triggers a periodic human checkpoint.

E Prompt-Graph Conventions

E.1 Stable node IDs

Prompt node IDs should be short, stable, and designed for graph evolution. A useful convention is integer batches with branch suffixes: P10, P11A, P11B, P12; repair nodes may use P11R1, and verification nodes P11V. IDs should not be renumbered merely to make the final graph visually contiguous because old logs and citations may already reference them.

E.2 Recommended node fields

The durable Plan or machine representation may include:

Field	Meaning
ID	stable prompt/run node identifier
Objective	bounded result the node should produce
Requests	DIC Request IDs served
Depends on	prerequisite nodes/artifacts
Parallel group	nodes eligible for concurrent execution
Owner/profile	Main/Theory/Experiment or bounded subagent/model preference
Read set	authoritative inputs required
Write scope	permitted outputs/mutations
Evidence	raw/test/citation/artifact outputs required
Local acceptance	node-level success criterion
Merge target	later node that integrates this result
Conditional branch	rule that decides whether follow-on nodes exist

Not every field must be repeated verbatim in Markdown if the meaning is obvious from a batch description, but machine materialization should be able to recover equivalent semantics.

E.3 Parallel batch example

A high-value research batch can be designed to reduce anchoring:

```
Batch T3 -- candidate theorem
P31A [Theory]      positive derivation
P31B [Theory]      hostile counterexample search
P31C [Research]    primary literature / nearest analogue search

parallel_group = T3-independent

P32 [Theory/Main] reconcile P31A/P31B/P31C
  if theorem survives and is material -> P33V fresh independent audit
  if theorem fails                    -> P33N strongest obstruction/negative
    result
  if novelty uncertain                -> P33L deeper literature audit
```

The critical feature is not the letters themselves. It is the explicit independence and merge structure.

E.4 Implementation batch example

For repository work:

```
P41A implement schema/runtime changes (write: aw/, schema/)
P41B update tests/fixtures             (write: tests/ only)
P41C update docs/templates             (write: docs/, templates/)

P42 integrate branches
P43 run unit + migration + recovery tests
P44 repair only failing surfaces
P45 release validation and report merge
```

Parallel implementation is safe only when write scopes or merge ownership are explicit.

F Report-Surface Reference Profiles

The ten-file maximum is a cap, not a target. The following profiles are examples of compact report design.

F.1 Theory

```
REPORT.md
RAW_RESULTS.jsonl
CLAIM_STATUS.json
AUDIT_RESULTS.jsonl
LITERATURE_MATRIX.csv
MANIFEST.json
```

`RAW_RESULTS.jsonl` may collate decisive derivation/counterexample outputs with source paths and hashes. `CLAIM_STATUS.json` is a compact machine view of claim classifications. The full literature PDFs, search logs, and long model outputs remain outside reports.

F.2 Experiment

```
REPORT.md
RAW_RESULTS.jsonl
RAW_RUNS.jsonl
RAW_GRADER_RESULTS.jsonl
RAW_METRICS.csv
PROTOCOL.json
ANALYSIS.csv
MANIFEST.json
```

The raw files preserve per-run/seed records. `ANALYSIS.csv` can contain derived estimates if clearly labelled as processed; it does not replace the raw surfaces.

F.3 Software/runtime

```
REPORT.md
RAW_RESULTS.jsonl
TEST_RESULTS.csv
BUILD_RESULTS.txt
FILE_CHANGES.json
MANIFEST.json
```

F.4 Publication/release

```
REPORT.md
RAW_RESULTS.jsonl
RULE_CHECKS.json
BUILD_RESULTS.txt
CITATION_AUDIT.csv
ARTIFACT_HASHES.json
MANIFEST.json
```

The report directory must not grow `raw/`, `final/`, or `archive/` subdirectories to evade the file cap.

G State and Lineage Field Guide

G.1 Plan-level state

Important Plan fields include identifiers and Goal text; approval status/actor/time; checkpoint interval and completed-since-checkpoint count; stable Phase order; active Phase identifier; and revision metadata. The Phase dictionary is keyed by stable Phase ID. The exact machine keys include `id`, `title`, `goal`, `status`, `approved_at`, `approved_by`, `checkpoint_interval`, `completed_since_checkpoint`, `phase_order`, and `active_phase_id`.

G.2 Phase-level state

Important Phase fields include `id`, `title`, `objective`, `owner`, `materiality`, `dependencies`, `status`, DIC and EPS identifiers/paths, Action identifiers/paths, verification records, acceptance records, evidence pointers, timestamps, lane policy, DIC-view policy, report policy, and DIC manifest path/hash.

The machine index should not embed every long prompt or raw output. Those objects remain addressable by path. This keeps `STATE.json` small enough to inspect while preserving full lineage externally.

G.3 Lineage interpretation

A complete execution record should allow the following questions to be answered:

- Which approved Goal/Plan authorized this artifact?
- Which Phase and Request did the work serve?
- Which DIC version/hash governed it?
- Which durable Plan node was instantiated?
- Which EPS attempt and prompt/run produced it?
- Which primitive actions/tools changed state?
- Which raw evidence supports acceptance?
- Which later artifacts depend on it?
- Was it repaired/superseded, and if so by what?

The purpose of lineage is not bureaucratic completeness; it is to make recovery, local invalidation, and scientific audit possible.

H Failure-Response Matrix

Failure	Default classification	First response	Escalate when
Tool command error	local execution	inspect stderr/state, bounded repair/retry	repeated failure implies architecture/resource change
Missing file/context	local execution	recover authoritative path/read set	source-of-truth ambiguity is material
Proof step fails	Theory repair	counterexample and assumption audit; new EPS	publication objective or frozen claim target changes materially
Novelty claim contradicted	Theory replan	downgrade/reframe claim, update ledger	venue/publication objective materially changes
Experiment parser bug	Experiment repair	correct parser, preserve raw input, re-run affected cone	frozen protocol/estimand must change
Confirmatory gate fails	scientific evidence	retain negative result, run predeclared analysis	proposed post-hoc redesign changes objective/claim materially
Verifier rejects artifact	repair/replan	address cited evidence defect, preserve rejection	disagreement concerns authority or release decision

Hidden dependency found	replan/locality widening	invalidate broader cone, update Architecture/Plan	structural architecture/Plan assumptions change
Report cap exceeded	deterministic repair	merge by semantic role, remove superseded copies	never by hiding files in subfolders
External API unavailable	local/block	retain evidence, use allowed fallback if equivalent	fallback changes data/method/cost/authority
Credential required	material	stop before access	always requires explicit authority
Paid compute required	material	estimate need and alternatives	always before purchase/paid run
Submission click/API	material	prepare artifact and checks only	external submission requires human confirmation

I Compatibility Map for v5.6.31 Concepts

v5.6.31 concept	v5.6.35 treatment	Note
Goal Mode / Structured Mode	compatibility profiles, not canonical hierarchy	useful legacy initialization styles; new work uses Plan/Phase/DIC
Persistent Orchestrator	orchestration capability of active workstream	Orchestrator alias maps to Main for compatibility
Researcher role Worker	research capability/lane bounded subagent/prompt-run executor	normally owned by Theory or Experiment does not acquire semantic authority
Verifier after many small units	local/milestone/independent verification levels	independent review reserved for material boundaries
Phase as large directory with many fixed folders	core Phase plus lazy optional lanes	reports remains mandatory
Single execution prompt stack	DIC Plan + one or more disposable EPS instances	durable graph separated from attempt-specific instantiation
Minor always-present machinery	off auto on lane	default lazy materialization
Many status/handoff reports	central state + one compact flat report surface	raw evidence preserved separately
Human approval at frequent execution boundaries	Plan approval + periodic checkpoints + immediate escalation	prompt IDs are not approval gates

The migration goal is not to declare the older design wrong. Several v5.6.31 mechanisms—prompt graphs, Minor work, explicit verifier prompts, execution-intensity guides, and rich phase artifacts—remain useful. Version 5.6.35 relocates them so they no longer force a human or persistent-session boundary at the same granularity as machine execution.

J Example Bounded Prompts

J.1 Theory derivation node

Outcome:

Determine whether the dependency-local recovery proposition holds under complete declared read/write dependencies, and state the strongest result.

Authority:

Read DIC/README.md, DIC/REQUESTS.md, DIC/PLAN.md, THEOREM_MAP.md, and the cited runtime semantics. Frozen empirical facts override intuition.

Constraints:

Do not strengthen the claim beyond explicit assumptions. Actively search for hidden-state counterexamples. Do not edit publication prose yet.

Evidence/success:

Return a formal statement or obstruction, proof/counterexample, assumption list, failure domain, and claim-ledger update candidate.

Autonomy:

Choose the derivation route. Escalate only if the Phase objective itself must materially change.

J.2 Experiment run node

Outcome:

Execute the frozen C3 condition on the assigned instance/seed batch.

Authority:

EXPERIMENT_PROTOCOL.md, TEST_MATRIX.md, GRADER_SPEC.md.

Constraints:

Do not change prompts, model, budgets, grader, exclusion rules, or fault injection. Preserve failures. No manual relabeling.

Evidence/success:

Write one raw run record per instance, grader output, token/time metadata, artifact hashes, and termination reason. Validate expected record counts.

Autonomy:

Repair deterministic infrastructure failures only when the protocol is unchanged; otherwise stop and record a protocol-change escalation.

J.3 Independent release audit node

Act as a fresh independent reviewer. You did not author the artifact. Read the release DIC, compact report surfaces, raw evidence required by material claims, and the candidate package/manuscript. Check rule compliance, scientific claim/evidence linkage, build/test status, anonymity where applicable, and unresolved escalations. Return accept, repair, replan, or escalate with exact evidence. Do not silently repair or submit externally.

References

Umut A. Acar. *Self-Adjusting Computation*. Morgan & Claypool, 2009.

Adrien Bardes, Quentin Garrido, Jean Ponce, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-jepa: Latent video prediction for visual representation learning. In *International Conference on Machine Learning*, 2024.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, and Torsten Hoefer.

- Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *International Conference on Machine Learning*, 2024.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations*, 2024.
- John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, 2024.
- Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. Program dependence graphs and their use in optimization. In *ACM Transactions on Programming Languages and Systems*, 1987.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.
- Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.
- Yann LeCun. A path towards autonomous machine intelligence. *OpenReview preprint*, 2022.
- Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: A research direction. *arXiv preprint arXiv:1811.07871*, 2018.
- Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for mind exploration of large scale language model society. In *Advances in Neural Information Processing Systems*, 2023.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. In *International Conference on Learning Representations*, 2024.
- Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. In *Proceedings of the ACM on Programming Languages*, 2018.
- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Aws Albarghouthi, and Frederic Sala. Slopcodabench: Measuring code erosion under iterative specification refinement. *arXiv preprint arXiv:2603.24755*, 2026. URL <https://arxiv.org/abs/2603.24755>.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, and Tushar Khot. Adapt: As-needed decomposition and planning with language models. In *Findings of the Association for Computational Linguistics: NAACL*, 2024.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Toolemu: Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Terminal-Bench Science Team. Terminal-bench science: Evaluating ai agents on computational workflows in the natural sciences. <https://github.com/harbor-framework/terminal-bench-science>, 2026.
- Zhenhailong Wang, Shaoguang Liu, Yixin Chen, Zhaoran Zheng, Mingyuan Wang, Kenji Kawaguchi, and Huaxiu Yao. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language models. *arXiv preprint*, 2024.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tenyue Hua, Zhoujun Cheng, Dong-Suk Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.
- Binfeng Xu, Zhendong Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*, 2024a.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024b.